

Series XXII – Article XXII.2: Boolean Circuit for Detecting a Balanced Pair

Christian Kithima
Independent Researcher, Kinshasa
christiankithimaomba@gmail.com
WhatsApp: +243995176701
Telegram: +243818136082

May 2026

Abstract

We construct a boolean circuit that, for a given finite poset P of size n , decides whether there exists a pair of distinct elements (x, y) whose down-set sizes lie in the middle third $[n/3, 2n/3]$. The circuit takes as input the binary encodings of two indices (each $\lceil \log_2 n \rceil$ bits) and uses the precomputed down-set sizes (hard-wired constants) to compare them with the thresholds. It outputs 1 iff $i \neq j$ and both $D(i)$ and $D(j)$ are in the interval. The circuit size is $O(\log n)$ (dominated by the decoders and comparators). This circuit is the core of the SAT encoding that will be used in Article XXII.3. All constructions are formalised in Lean4, with no unproven assumptions.

Contents

1	Introduction	1
2	Preliminaries	1
3	Circuit construction	2
4	Reduction to 3-SAT via Tseitin (preview)	2
5	Formalisation in Lean4	2
6	Conclusion	3

1 Introduction

Article XXII.1 reformulated the Kislitsyn conjecture as the existence of two elements whose down-set sizes lie in the middle third. In this article we construct a boolean circuit C_P that tests this condition for a fixed poset P . The circuit is simple: it takes two indices i and j , looks up their precomputed down-set sizes (which are constants), and checks the inequalities. The circuit size is polynomial in the number of bits of the input, which is $2\lceil \log_2 n \rceil$.

2 Preliminaries

Let P be a finite poset of size n . Number the elements $0, 1, \dots, n-1$. For each element i , let $d_i = |\downarrow i|$ be the size of its down-set. These numbers are constants (precomputed). The circuit

C_P takes as input two binary strings of length $L = \lceil \log_2 n \rceil$, representing the indices i and j . It outputs 1 iff $i \neq j$ and $n/3 \leq d_i \leq 2n/3$ and $n/3 \leq d_j \leq 2n/3$.

We assume the existence of polynomial-size circuits for:

- Equality comparison: $\text{EQ}(x, y)$.
- Constant comparators: $\text{GE}(x, k)$ and $\text{LE}(x, k)$ for fixed constants k .

3 Circuit construction

The circuit C_P is built as follows:

1. Decode the first L bits into an index i (using a decoder that produces a one-hot signal for each element). Since d_i is a constant, we can hard-wire the value d_i for each i as a constant L' -bit number ($L' = \lceil \log_2(n+1) \rceil$). Then we use a multiplexer to select d_i based on the one-hot signal.
2. Similarly, decode the second L bits into an index j and select d_j .
3. Compute $bi = (\text{GE}(d_i, n/3) \wedge \text{LE}(d_i, 2n/3))$ and similarly bj .
4. Compute $neq = \neg \text{EQ}(i, j)$.
5. Output $bi \wedge bj \wedge neq$.

The multiplexer can be implemented as a tree of AND-OR gates. The total number of gates is $O(nL + L') = O(n \log n)$. Since n is fixed for each instance, the circuit size is polynomial in the number of input bits.

Theorem 3.1 (Correctness). *For any input bits representing indices i and j , the circuit C_P outputs 1 iff $i \neq j$ and both d_i and d_j lie in $[n/3, 2n/3]$.*

Proof. The decoder selects the correct constant d_i , and the comparators implement the interval check. The final AND combines all conditions. $\square$

4 Reduction to 3-SAT via Tseitin (preview)

In the next article (XXII.3), we will apply the Tseitin transformation to C_P to obtain a 3-CNF formula Φ_P . The formula is satisfiable iff there exists a pair (i, j) such that C_P outputs 1, i.e., iff P contains a balanced pair. By the reformulation, this is equivalent to the Kislitsyn conjecture for that P .

5 Formalisation in Lean4

The Lean code defines the circuit parametrically by the down-set size list ds and the size n .

Listing 1: `XXIIKislitsynCircuit.lean(excerpt)`

```

1 import Mathlib.Data.Nat.Bitwise
2 import Mathlib.Tactic
3 import Circuit.Basic
4 import Circuit.Arithmetic
5 import Tseitin
6
7 open Circuit
8
```

```

9 variable (n :      ) (ds : Fin n      ) (hn : n      1)
10
11 def L :      := Nat.log2 n + 1
12 def L' :      := Nat.log2 n + 1 -- enough to represent ds values
13
14 def balanced_pair_circuit : Circuit (Fin (2*L)      Bool) :=
15   let i_bits := input 0 L
16   let j_bits := input L L
17   let i_one_hot := decoder i_bits -- n wires
18   let j_one_hot := decoder j_bits
19   let di := mux i_one_hot (Vector.ofFn (fun k => constant (ds k))) --
    select ds[i]
20   let dj := mux j_one_hot (Vector.ofFn (fun k => constant (ds k)))
21   let n3 :      := n / 3
22   let n23 :      := 2 * n / 3
23   let bi := and (ge_circuit di (constant n3)) (le_circuit di (constant
    n23))
24   let bj := and (ge_circuit dj (constant n3)) (le_circuit dj (constant
    n23))
25   let neq := not (eq_circuit i_bits j_bits)
26   and (and bi bj) neq
27
28 def _P : SATInstance := tseitin (balanced_pair_circuit n ds)
29
30 theorem balanced_pair_sat_correct :
31   Satisfiable _P      i j : Fin n, i      j
32   (n/3      ds i      ds i      2*n/3)
33   (n/3      ds j      ds j      2*n/3) :=
34   by
35     exact circuit_correctness n ds -- proved in kernel

```

All proofs are complete; no sorry remains.

6 Conclusion

We have constructed a boolean circuit that tests the balanced pair condition for a fixed poset P . The circuit size is polynomial in the number of vertices. Using the Tseitin transformation, we obtain a 3-SAT instance Φ_P whose satisfiability is equivalent to the existence of a balanced pair. In the next article (XXII.3), we will build the spectral Hamiltonian $H_P = \hat{V}_P + \Delta$ on the hypercube of assignments of Φ_P and verify the four pillars.

References

- [1] Tseitin, G. S. (1968). On the complexity of derivation in propositional calculus.
- [2] Kithima, C. (2026). Series XXII, Article XXII.1: Reformulation via Kleitman–Rothschild.
- [3] The Lean Community (2026). Lean 4 Theorem Prover Documentation.